Cycles and Functionality in cvc5's Relations Solver

`rel.cyclic`, `rel.is-functional` and their calculus

the pull request to cvc5 main (branch `cycle-ext-cyclic`)

October 2026

Outline

1. Motivation
2. What is new compared to cvc5 main
3. The calculus
4. Strategy
5. Last call and model construction
6. Results
7. Evidence for every rule
8. Summary

Relational specifications need acyclicity and functionality

Axiomatic memory models allow an execution iff some union of relations is acyclic, e.g. sequential consistency: $po \cup rf \cup co \cup fr$ is acyclic.
Alloy-style specifications constrain relations to be (partial) functions.

In cvc5 main both properties must be stated with quantifiers:

$$\forall x.\ (x, x) \notin TC(R)$$
$$\forall x\, y\, z.\ (x, y) \in R \wedge (x, z) \in R \Rightarrow y = z$$

and the negation of acyclicity needs an existential witness. Instantiation (E-matching or finite model finding) has to find the right instances.

With this PR, two predicates that the relations solver reasons about directly:

```
; a forbidden execution: a cycle
(assert (rel.cyclic
  (set.union po (set.union rf
    (set.union co fr)))))
; po is acyclic and functional
(assert (not (rel.cyclic po)))
(assert (rel.is-functional po))
```

No quantifier for acyclicity or functionality; a calculus of inference rules instead (this talk).

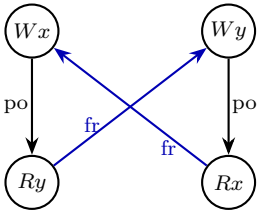
The benchmarks: completeness of litmus-test templates

13 memory-model benchmarks (relational-solver-benchmarks): SC, SC per location (plsc), TSO; one and two threads; variants `just-rf` and `no-ltts`.

Each asks: *is there an execution that violates the memory model but that no litmus-test template forbids?*

- `unsat`: the templates are complete (10 files);
- `sat`: cvc5 prints such an execution (3 files).

The violation is a cycle: `(rel.cyclic (set.union po rf co fr))`; the templates and the model are acyclicity and functionality constraints on the derived relations.



store buffering: a cycle of po ∪ fr, forbidden by SC

The changes at a glance

component	change
kinds, API	`RELATION_CYCLIC` (`rel.cyclic`), `RELATION_IS_FUNCTIONAL` (`rel.is-functional`), `RELATION_RCLOSURE`, `RELATION_RTCLOSURE` (`rel.rclosure`, `rel.rtclosure`); parser, printer, CPC signature
type rules	a binary relation whose columns have the same sort (cyclic, closures); any binary relation (functional)
rewriter	evaluation and elimination rules (constants, empty set, singletons, unions)
relations solver	the calculus: cycle witness, minimality, acyclicity, anchors, closure induction, functionality, lazy closure splits, grounding conflicts
strategy	a new full-effort step, three last-call steps, `needsCheckLastEffort`
options	`--rels-functional-mode`, `--rels-acyclic-anchor`, `--rels-acyclic-backward-chords`, `--rels-tc-subset`, `--rels-tc-down-lazy`, `--rels-acyclic-hammer`
defaults	MiniSat under `--sets-exp` (justification heuristic), except for sygus-style problems
fixes	closure rules (edge explanations, conclusions on the tuple of the explanation); no redundant facts in the sets solver
tests, docs	53 regression files; `docs/theories/sets-and-relations.rst`

Against cvc5 main (a3f47bdffd): 28 source files, +3305/ − 131 lines; tests +1958 lines.
`scripts/` (experiment scripts, these slides) is temporary.

New predicates

`(rel.cyclic R)`, R : `(Relation T T)`

$$\mathsf{cyclic}(R) \iff \exists x.\ (x, x) \in R^+$$

Acyclicity is `(not (rel.cyclic R))`.

e.g. `(rel.cyclic (set.union R S))` holds for $R = \{(a, b)\}$, $S = \{(b, a)\}$; `(rel.cyclic R)` and `(rel.cyclic S)` do not.

`(rel.is-functional R)`, R : `(Relation T1 T2)`

$$\mathsf{fun}(R) \iff \forall x\, y\, z.\ (x, y), (x, z) \in R \Rightarrow y = z$$

Right-unique, not necessarily total.

`(rel.rclosure R)`, `(rel.rtclosure R)`

R plus (x, x) for every x in its domain or range; and that union with R^+. Eliminated by rewriting when R is built from $\emptyset$, singletons and unions; otherwise the solver answers `unknown`.

Why a predicate and not a definition?
The definitions need a quantifier (or the universe set, which cvc5 does not interpret as all elements). As predicates, the solver builds *witnesses* and *explanations* for them, and uses their structure (unions, closures, joins) in its rules.

Rewrite rules: what holds in every context

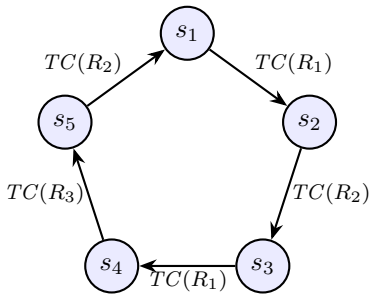
The rewriter only replaces a term by an *equivalent* term, independently of the assertions:

term	rewritten to	why
`(rel.cyclic ∅)`	`false`	no pair at all
`(rel.cyclic {(x,y)})`	`(= x y)`	only a self-loop
`(rel.cyclic` c`)`, c constant	`true` / `false`	evaluate c^+
`(rel.is-functional X)`, X: ∅, `{t}`, `(rel.iden A)`	`true`	at most one image
`(rel.is-functional` c`)`, c constant	`true` / `false`	first components
`(rel.rclosure {(x,y)})`	$\{(y,y)\} \cup (\{(x,x)\} \cup \{(x,y)\})$	definition
`(rel.rclosure (set.union A B))`	`(rel.rclosure A)` ∪ `(rel.rclosure B)`	union
`(rel.rtclosure R)`	`(rel.rclosure R)` ∪ `(rel.tclosure R)`	definition

Everything else is done by *inference rules* in the relations solver (next section).

Notation

- A configuration: the asserted literals and the equivalence classes of the equality engine; $t \sim t'$: equal in the current context.
- A rule $\dfrac{P_1 \quad \dots \quad P_n}{C}$ sends the *lemma* $P_1 \wedge \dots \wedge P_n \Rightarrow C$, a valid clause whose premises are the explanation (used by conflict analysis, unsat cores and proofs).
- Fresh symbols are skolems, cached by their key, so each is created once.
- The *cycle witness* of $\mathsf{cyclic}(R_1 \cup \dots \cup R_k)$: elements $s_1, \dots, s_l$ with $s_l = s_1$ ($l - 1$ edges), each edge in some $TC(R_j)$.



$l = 6,\ s_6 = s_1$

Cyclicity: creating and unrolling the witness

For an asserted $\mathsf{cyclic}(R)$ (fresh s, s_1, l, s_{i+1}):

$$\frac{\mathsf{cyclic}(R) \quad R \text{ not a union}}{(s,s) \in TC(R)}\ \textsc{Cycle-Def} \qquad \frac{\mathsf{cyclic}(R_1 \cup \cdots \cup R_k)}{1 < l}\ \textsc{Inst-Cycle}$$

$$\frac{i < l}{(s_i, s_{i+1}) \in TC(R_1) \ \vee \ \ldots \ \vee \ (s_i, s_{i+1}) \in TC(R_k)}\ \textsc{Unroll}$$

$$\frac{\mathsf{cyclic}(R) \quad i \le l}{i < l \ \vee \ (l = i \wedge s_i = s_1)}\ \textsc{Split-Len} \qquad \frac{\mathsf{cyclic}(R') \quad R' \text{ has the operands of } R}{\mathsf{cyclic}(R)}\ \textsc{Same-Operands}$$

- The operands of nested unions are flattened: the witness of $\mathsf{cyclic}(rf \cup (po \cup co))$ alternates between $TC(rf)$, $TC(po)$, $TC(co)$.
- One Unroll per check round; at last call the witness is completed to the value of l in the candidate model.
- Unroll needs no $\mathsf{cyclic}(R)$ premise: if $\mathsf{cyclic}(R)$ is false, l is unconstrained and $l \le 1$ makes every instance vacuous.

Minimality: the witness is a shortest cycle

The witness can be a shortest cycle of $TC(R_1) \cup \cdots \cup TC(R_k)$ ($q < r \leq l$, every R_j):

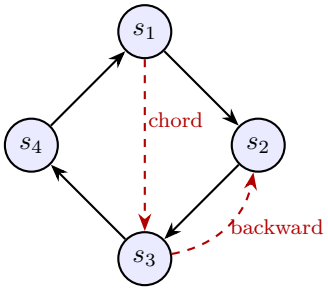
$$\frac{\mathsf{cyclic}(R) \quad i < l}{\neg\big((s_{i-1}, s_i) \in TC(R_j) \wedge (s_i, s_{i+1}) \in TC(R_j)\big)}\ \textsc{Alternate}$$

$$\frac{\mathsf{cyclic}(R) \quad l = i + 1}{\neg\big((s_i, s_{i+1}) \in TC(R_j) \wedge (s_1, s_2) \in TC(R_j)\big)}\ \textsc{Alt-Close}$$

$$\frac{\mathsf{cyclic}(R) \quad r \leq l \quad q < r - 1}{(s_q, s_r) \notin TC(R_j)}\ \textsc{Chord}$$

$$\frac{\mathsf{cyclic}(R) \quad r \leq l \quad (q, r) \neq (1, l)}{s_q \neq s_r}\ \textsc{Distinct}$$

Backward (`--rels-acyclic-backward-chords`): $(s_r, s_q) \notin TC(R_j)$ if it closes a shorter cycle; lets a totality axiom (e.g. *po*) bound the witness.



$l = 5$: both dashed edges would give a shorter cycle

Acyclicity

For an asserted $\neg\mathsf{cyclic}(u)$:

$$\frac{(a,b) \in TC(R) \qquad \neg\mathsf{cyclic}(u) \qquad u \sim R}{a \neq b}\ \textsc{Acyclic-Down} \qquad\qquad \frac{\neg\mathsf{cyclic}(u)}{u \subseteq TC(u)}\ \textsc{Register}$$

- Acyclic-Down turns every reflexive closure membership into a conflict. The existing closure rules produce those memberships:

$$\frac{(a,b) \in R \qquad (b,c) \in TC(R)}{(a,c) \in TC(R)}\ \text{TC-Up}$$

$$\frac{(a,b) \in TC(R)}{(a,b) \in R \vee \big((a,k_1),(k_2,b) \in R \wedge (k_1 = k_2 \vee (k_1,k_2) \in TC(R))\big)}\ \text{TC-Down}$$

- Register is valid and does nothing logically: it makes $TC(u)$ a term of the solver even if the input never mentions it, so that a cycle of u is found.
- The conclusion is stated on the tuple of the explanation, not on its representative (the lemma is context-independent).

Cyclic meets acyclic: anchor rules

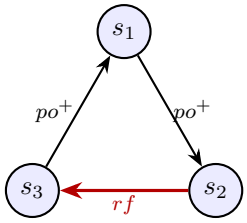
A cycle of R all of whose edges lie in $TC(S)$ would be a cycle of S:

$$\frac{\mathsf{cyclic}(R) \qquad \neg\mathsf{cyclic}(S)}{t \in R' \wedge t \notin TC(S)}\ \textsc{Inclusion}$$

$$\frac{\mathsf{cyclic}(R) \qquad \neg\mathsf{cyclic}(S) \qquad l = N}{\bigvee_{i<N}(s_i, s_{i+1}) \notin TC(S)}\ \textsc{Branch}$$

t fresh; R': the operands of R *not* syntactically in $TC(S)$ (e.g. $S \cap Y$, $TC(S)$, $S;S$ are). If there is none, the lemma is `false`.

Branch is sent at last call, with N the value of l in the candidate model; with $l = N$ in its premises the listed edges are the whole cycle. `--rels-acyclic-anchor`: `branch` (default), `inclusion`, `both`, `none`.



$\mathsf{cyclic}(po \cup rf)$, $\neg\mathsf{cyclic}(po)$:
Inclusion gives $t \in rf$, $t \notin po^+$

Closure induction (`--rels-tc-subset`)

$TC(R)$ is the least transitive relation that contains R:

$$\frac{(a,b) \in TC(R) \qquad R \subseteq S \qquad S;S \subseteq S}{(a,b) \in S}\ \text{TC-Subset}$$

The transitive supersets S recognised in the current context:

superset S	recognised by	example
$TC(X)$, R syntactically in $TC(X)$	the shape of R	$R = po^+ \cap loc$, $S = po^+$
S transitive, $R \subseteq S$ asserted	$(R \cup S) = S$ in the context	$S = A \times B$, a closure, $J; J^T$ with $\mathsf{fun}(J)$
R itself	$(R;R) \subseteq R$ asserted	co transitive: $TC(co) = co$
R itself	$R \subseteq A \times B$, $A \cap B = \emptyset$	fr from reads to writes: $TC(fr) = fr$
X, transitive	$R = X \cap Y$ or $R = X \setminus Y$	

No fresh element: where it applies, one lemma replaces an unbounded descent of TC-Down splits, each of which introduces two elements. This is what the hand proofs of the benchmarks use ("every ghb path is in po^+").

Functionality (`--rels-functional-mode`)

$$\frac{\mathsf{fun}(R) \qquad (x,y) \in S \qquad S \sim R}{y = f_R(x)}\ \textsc{Fun-UF} \qquad \frac{\mathsf{fun}(R) \qquad (x_1,y_1) \in R \qquad (x_2,y_2) \in R}{x_1 = x_2 \Rightarrow y_1 = y_2}\ \textsc{Fun-Pairs}$$

$$\frac{\neg\mathsf{fun}(R)}{(k_1,k_2) \in R \wedge (k_1,k_3) \in R \wedge k_2 \neq k_3}\ \textsc{Not-Fun}$$

- `uf` (default): a fresh function f_R; linear in the members, congruence does the rest: $(a,b),(a,c) \in R$ give $b = f_R(a) = c$. `pairs`: quadratic, no function symbol (used when the logic has no UF).
- Feeds closure induction: $J; J^T$ is transitive if J is functional ($J^T; J$ relates only equal elements), e.g. $\mathit{same\ location} = \mathit{address}; \mathit{address}^T$.

Splitting closures and joins: eager, lazy, or not at all

Eager (cvc5 main): TC-Down for every closure membership that the known members do not explain, and Join-Split for every join membership, each with fresh elements. With finite model finding this floods the model with elements.

$$\frac{(x,z) \in A;B}{(x,k) \in A \wedge (k,z) \in B}\ \textsc{Join-Split} \qquad \frac{(x,y) \in A \quad (y,z) \in B}{(x,z) \in A;B}\ \textsc{Join-Compose}$$

Lazy (`--rels-tc-down-lazy`): TC-Down only at last call, and only for memberships that the known members of R do not explain. Unsat proofs that never unfold a closure (closure induction) introduce no element.

Not at all (`--rels-acyclic-hammer`, for unsat problems): no TC-Down and no Join-Split. At last call, a closure or join membership that the known members do not explain is a conflict (Ground-TC, Ground-Join; see *Grounding at last call*).

Lazy and hammer move element introduction for closures and joins out of the full-effort check: the last call either introduces them only where needed (lazy) or refutes the candidate model instead (hammer).

Why inference rules and not rewrite rules?

The rewriter

maps a term to an *equivalent* term:

- the same in every context (no access to assertions or to the model);
- no fresh symbols, no case splits;
- terminating, and applied to the input and to every lemma.

It gets the rules on the term alone: $\mathsf{cyclic}(\emptyset) = \bot$, $\mathsf{cyclic}(\{(x, y)\}) = (x = y)$, constants, closure elimination.

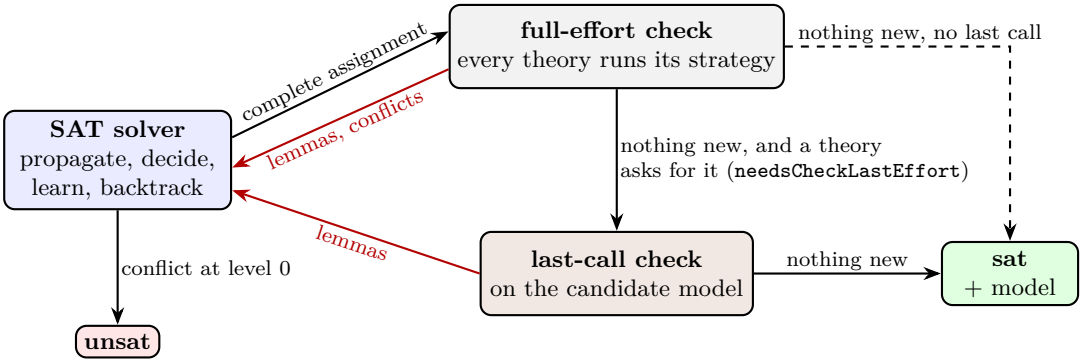
Rewriting $\mathsf{cyclic}(R)$ to its definition needs a quantifier, or $TC(R) \cap iden(univ) \neq \emptyset$, which is wrong in cvc5 (the universe is not all elements) and hides the structure the rules use.

the rules need	e.g.
a **witness** (an existential: fresh symbols)	Cycle-Def, Inst-Cycle, Unroll, Inclusion, Not-Fun
case splits for the SAT solver	Unroll picks an operand, Split-Len continues or closes
other asserted facts	TC-Subset ($R \subseteq S$, S transitive), anchors ($\mathsf{cyclic}(R)$ and $\neg\mathsf{cyclic}(S)$), Fun-UF (a membership)
equivalence classes	$S \sim R$, $u \sim R$: premises that hold now, not always
the candidate model	Branch ($l = N$), Ground-TC, Ground-Join
unbounded unfolding	a witness of any length, one element per round

Soundness

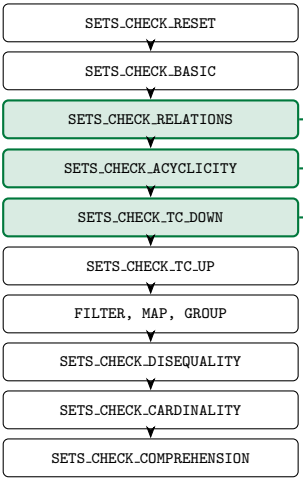
- **Valid lemmas**, consequences of the definitions and the premises: the closure rules, Acyclic-Down, Register, TC-Subset, Fun-Pairs, and the grounding conflicts (thanks to the distinctness premise).
- **Lemmas about skolems**, satisfiability-preserving: any model of the input extends to one in which the witness is a shortest cycle of the operands' closures, f_R maps every element to its image, etc. This covers Cycle-Def, Inst-Cycle, Unroll, Split-Len, Alternate, Alt-Close, Chord, Distinct, Backward, Inclusion, Fun-UF, Not-Fun. Branch follows from them (thanks to $l = N$). The last call sends instances of the same rules (see *Is introducing elements at last call sound?*).
- **Incompleteness is reported**, so cvc5 answers `unknown` instead of a wrong `sat`, when
  - the length of a witness has no practical value in the model;
  - a `rel.rclosure` term is not eliminated by rewriting.
- **Proofs.** The lemmas are trusted theory lemmas; their CPC proofs check with ethos (`rel.cyclic`, `rel.is-functional` are in the CPC signature).

Where the rules run: cvc5's DPLL(T) loop



- The sets theory asks for a last call (`needsCheckLastEffort`) if a cycle witness exists, or under `--rels-tc-down-lazy` / `--rels-acyclic-hammer` if closures or joins occur.
- Lemmas are permanent clauses; the witness and the caches are rolled back on backtracking (context-dependent).

The sets strategy at full effort



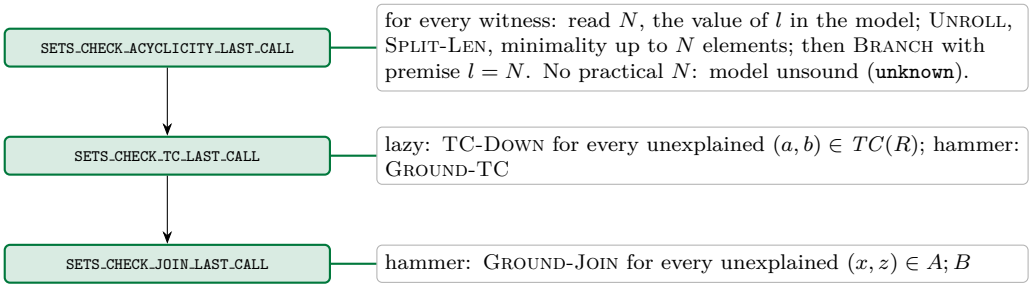
relations: collect the asserted literals and terms; CYCLE-DEF, INST-CYCLE, SAME-OPERANDS, REGISTER, NOT-FUN; per membership the join, product, transpose and identity rules, ACYCLIC-DOWN; FUN-UF / FUN-PAIRS

new step: per witness one UNROLL, SPLIT-LEN, ALTERNATE, ALT-CLOSE, CHORD, DISTINCT, BACKWARD, INCLUSION; BRANCH for self-loops

closures: TC-DOWN (deferred if lazy, skipped by the hammer) and TC-SUBSET per closure membership; no break: TC-UP runs in the same round

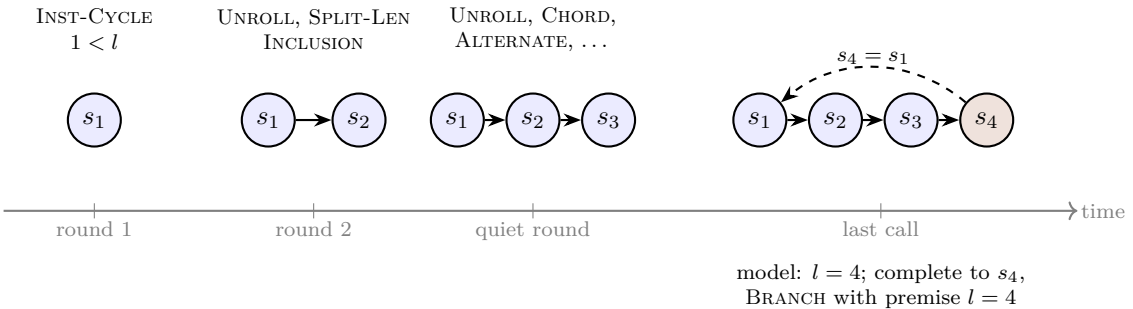
After each step (a *break*): if the step sent a lemma or a fact, the round stops. Facts only: the strategy runs again; a lemma: back to the SAT solver. Green: new or changed by this PR. TC: `TRANSITIVE_CLOSURE`.

Last call: the candidate model decides



- A step that sends a lemma ends the check; the SAT solver continues with it.
- Model-dependent rules belong here: at full effort the length of the witness is still symbolic, and the members of a relation are not final.

A cycle witness across the search



- Each round's lemmas go back to the SAT solver; a conflict may backtrack past the creation of s_3, which then disappears from the witness. It is recreated, as the same skolem, with lemmas that are already known: that round is quiet, and the last call runs.
- A witness that is too long for the problem is refuted by minimality and the anchors; a short one survives and its edges become the model's cycle.

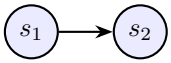
Last call: completing the witness

Algorithm 1: `checkAcyclicityLastCall`

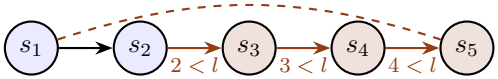
```
1  foreach witness s_1..s_c, l of an asserted
     cyclic(R_1 ∪ ··· ∪ R_k) do
2      N ← the value of l in the candidate model
3      if N is not a constant, or N > 10000 then
4          Len-Unknown: model unsound; continue
5      while c < N do
6          s_{c+1} ← the skolem cached for (R, c + 1)
7          send Unroll_c, Alternate_c, Alt-Close_c
8          c ← c + 1
9          send Split-Len_c, and Chord, Distinct,
10             Backward for the pairs up to s_c
11     if N ≥ 2 then
12         foreach asserted ¬cyclic(S) do
13             send Branch_N (once per R, S, N)
```

A lemma that was sent before is not sent again (the lemma cache survives backtracking).

current context: $c = 2$; model: $l = 5$



after the last call: $c = 5$



Split-Len$_5$: $l = 5 \Rightarrow s_5 = s_1$

- **Why** $c < N$: full effort adds one element per round, backtracking removes them. Re-unrolled with the same skolems, a round sends only known lemmas: it is quiet, and the last call runs. Each new edge (orange) has its own guard $i < l$.

The lemmas about the witness

Each lemma is *premise* $\Rightarrow$ *conclusion*, for the witness of $\mathsf{cyclic}(R)$, $R = R_1 \cup \cdots \cup R_k$, and every operand R_j:

rule	premise		conclusion
Inst-Cycle	$\mathsf{cyclic}(R)$		$1 < l$
Unroll$_i$	$i < l$		$(s_i, s_{i+1}) \in TC(R_1) \vee \cdots \vee (s_i, s_{i+1}) \in TC(R_k)$
Split-Len$_r$	$\mathsf{cyclic}(R) \wedge r \leq l$		$r < l \ \vee \ (l = r \wedge s_r = s_1)$
Alternate$_i$	$\mathsf{cyclic}(R) \wedge i < l$	$(i \geq 2)$	$\neg\big((s_{i-1}, s_i) \in TC(R_j) \wedge (s_i, s_{i+1}) \in TC(R_j)\big)$
Alt-Close$_i$	$\mathsf{cyclic}(R) \wedge l = i + 1$	$(i \geq 2)$	$\neg\big((s_i, s_{i+1}) \in TC(R_j) \wedge (s_1, s_2) \in TC(R_j)\big)$
Chord$_{q,r}$	$\mathsf{cyclic}(R) \wedge r \leq l$	$(q < r - 1)$	$(s_q, s_r) \notin TC(R_j)$
Distinct$_{q,r}$	$\mathsf{cyclic}(R) \wedge r \leq l$ [$\wedge\, r \neq l$ if $q = 1$]		$s_q \neq s_r$
Branch$_N$	$\mathsf{cyclic}(R) \wedge \neg\mathsf{cyclic}(S) \wedge l = N$		$(s_1, s_2) \notin TC(S) \vee \cdots \vee (s_{N-1}, s_N) \notin TC(S)$
Len-Unknown	l has no value ≤ 10000		model unsound: `unknown`, never `sat`

- The premises mention the term l, never its value. N only decides *how many* instances the last call sends, and appears only as the literal $l = N$ of Branch$_N$.
- s_i is the skolem cached for (R, i): full effort and the last call build the same element and lemma. The last call changes *when* a lemma is sent, not *what* it says.

Is introducing elements at last call sound? (1/2)

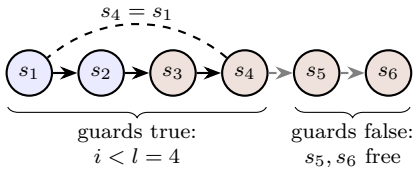
Claim: every model M of the input extends to a model M' that satisfies the lemmas about the witness *for all indices and all N at once*: whatever values of l the candidate models show, no model of the input is lost (no wrong `unsat`).

M' interprets only the skolems:

- cyclic(R) true in M: take a *shortest* cycle of $TC(R_1) \cup \cdots \cup TC(R_k)$ in M, with $L - 1$ edges. Set $l := L$, put $s_1, \ldots, s_L$ along the cycle ($s_L = s_1$), and let s_i, $i > L$, be any element.
- cyclic(R) false in M: $l := 1$, every s_i any element.

Every guard of an index beyond L ($i < l$, $r \leq l$, $l = N$) is false in M': the elements created for a larger value of l are unconstrained.

an earlier candidate model had $l = 6$, so a last call created $s_3, \ldots, s_6$; the final model has $l = 4$



- A shortest cycle exists if a cycle does; no edge of a $TC(R_j)$ shortcuts it.
- Other rules on the s_i (TC-Down, ...) are valid or pick their own fresh elements after the s_i.

Is introducing elements at last call sound? (2/2)

Every lemma holds in M' ($\mathsf{cyclic}(R)$ true in M, shortest cycle with $L-1$ edges, $G = TC(R_1) \cup \cdots \cup TC(R_k)$):

rule	index up to L: the guard holds, the conclusion too	index beyond L
Inst-Cycle	$1 < L$: the cycle has an edge	—
Unroll$_i$	$i < L$: (s_i, s_{i+1}) is an edge of the cycle, so it is in some $TC(R_j)$	$i < l$ false
Split-Len$_r$	$r < L$: $r < l$; $r = L$: $l = L$ and $s_L = s_1$	$r \leq l$ false
Alternate$_i$, Alt-Close$_i$	two consecutive edges in $TC(R_j)$ give, by transitivity, one edge of $TC(R_j)$: a cycle of G with one edge less	$i < l$, $l = i + 1$ false
Chord$_{q,r}$	$(s_q, s_r) \in TC(R_j)$ skips $r - q \geq 2$ edges: a shorter cycle (for $(1, L)$: a self-loop, 1 edge)	$r \leq l$ false
Distinct$_{q,r}$	$s_q = s_r$ closes the cycle $s_q..s_r$ of $r - q < L - 1$ edges	$r \leq l$ false
Branch$_N$	$N \neq L$: $l = N$ is false. $N = L$: if every edge were in $TC(S)$, S would be cyclic in M, but $M \models \neg\mathsf{cyclic}(S)$	—

$\mathsf{cyclic}(R)$ **false in** M: every premise contains $\mathsf{cyclic}(R)$, except Unroll$_i$, whose guard $i < l = 1$ is false.

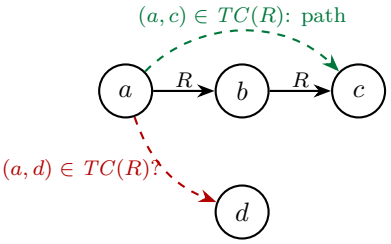
The other direction (no wrong `sat`): a quiet last call means that the model satisfies Unroll$_1$..Unroll$_{N-1}$ and Split-Len$_N$ with $l = N$: it has a cycle (see *Model construction*). Without a practical N: `unknown`.

Grounding at last call: the algorithm

Algorithm 2: last call: closures, joins

```
1  collect the members of every relation, by representatives
2  foreach membership (a, b) ∈ TC(R) do
3      if no path a → ··· → b in the graph of the known
        members of R then
4          if lazy then send TC-Down for (a, b)
5          else send Ground-TC                    (hammer)
6  if hammer then
7      foreach membership (x, z) ∈ A; B do
8          if no known (x, y) ∈ A with (y, z) ∈ B then
9              send Ground-Join
```

- Closure (`isTCReachable`): depth-first search from a over the members of R only.
- Join (`isJoinReachable`): for each known $(x, y) \in A$, look up (y, z) in the membership trie of B.



$R = \{(a, b), (b, c)\}$: (a, c) is explained, (a, d) is not.

- lazy: TC-Down adds $(a, k_1), (k_2, d) \in R$, $\ldots$;
- hammer: Ground-TC refutes this model: the SAT solver must change R, an equality, or the membership.

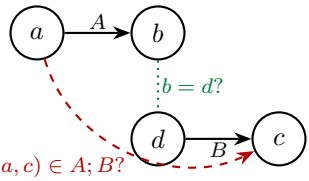
Grounding conflicts: GROUND-TC and GROUND-JOIN

$$\frac{(a,b) \in TC(R) \quad R = \{m_1, .., m_n\} \quad distinct(a, b, \vec{e})}{\bot}\ \text{GROUND-TC}$$

$$\frac{(x,z) \in A;B \quad A = \{m_1, .., m_n\} \quad B = \{p_1, .., p_o\} \quad distinct(x, z, \vec{e})}{\bot}\ \text{GROUND-JOIN}$$

$\{m_1, .., m_n\}$: the known members. $\vec{e}$: the representatives of the elements of the m_i, p_j (the equalities between terms and representatives are premises too).

Valid: under the premises, R is exactly the n pairs, over pairwise distinct elements, so $TC(R)$ is the set of paths of that graph, and the search found none from a to b. Likewise for $A;B$. No term is introduced: the SAT solver must change the model.



$(a, c) \in A;B$ with $A = \{(a, b)\}$, $B = \{(d, c)\}$:

- a, b, c, d distinct: the membership is impossible, the conflict is valid;
- $b = d$: the membership holds. The premise $distinct(a, c, b, d)$ is false, so the conflict does not exclude this model.

Without it, GROUND-JOIN refuted $b = d$ too: a wrong `unsat` (fixed in this PR; GROUND-TC had it).

Model construction: why `sat` is correct

How cvc5 builds the model (`collectModelValues`): every set equivalence class gets the union of the singletons of its known members. Relational terms are *not* recomputed, so `sat` is correct only if, when the last call is quiet, the member lists agree with the operators:

term / literal	every required member is there	every member is justified
$TC(R)$	TC-Up: the pairs of every path of R	TC-Down (full effort or lazy), or no unexplained member (hammer)
$A; B$	Join-Compose	Join-Split, or no unexplained member (hammer)
$\mathsf{cyclic}(R)$	quiet last call: $c \geq N$, and the model satisfies Unroll$_i$ ($i < N = l$) and Split-Len$_N$, so $s_1 \to \cdots \to s_N = s_1$ with every edge in some $TC(R_j)$, which is the closure of R_j in the model (rows above): R has a cycle. Self-loop form: $(s, s) \in TC(R)$.	
$\neg\mathsf{cyclic}(u)$	Register makes $TC(u)$ a term; TC-Up puts (a, a) into it for any cycle of the members of u; Acyclic-Down refutes that: u is acyclic in the model.	
$\mathsf{fun}(R)$, $\neg\mathsf{fun}(R)$	Fun-UF / Fun-Pairs: no $(x, y_1), (x, y_2)$ with $y_1 \neq y_2$; Not-Fun: two such members.	

Options and defaults

option	default	effect
`--rels-functional-mode=uf|pairs`	`uf`	how `rel.is-functional` is used
`--rels-acyclic-anchor=branch|inclusion|both|none`	`branch`	anchor rules
`--rels-acyclic-backward-chords`	off	Backward (minimality)
`--rels-tc-subset`	off	closure induction
`--rels-tc-down-lazy`	off	TC-Down only at last call
`--rels-acyclic-hammer`	off	no TC-Down / Join-Split; grounding conflicts
`--sets-exp` ⇒ MiniSat	on	SAT solver unless set by the user or the problem is sygus-style

All six new options are expert options, off or at their previous behaviour by default. The closure rules that need them (closure induction, lazy split, hammer) win on the benchmarks only in combination (next section); turning them on by default needs a broader evaluation (e.g. on cvc5/sets-and-relations-benchmarks).

Setup

name	options (all on the files without `:finite-model-find`)
default	`--finite-model-find` (as published)
FMF	default + `--e-matching --inst-when=full-delay --rels-acyclic-anchor=inclusion --rels-acyclic-backward-chords`
noFMF	`--rels-acyclic-anchor=inclusion --inst-when=full-delay --rels-acyclic-backward-chords`
stop-only	default + `--decision=stoponly`
FMF+R	FMF + R + `--rels-tc-down-lazy`
noFMF+R	noFMF + R + `--rels-tc-down-lazy`
noFMF+H	noFMF + R + `--rels-acyclic-hammer`

R = `--prenex-quant=none --rels-tc-subset`.

- 13 benchmarks with `rel.cyclic` and `rel.is-functional` (relational-solver-benchmarks e4a4f7e), release build of the branch (45502e064c), 300 s.
- **Virtual portfolio**: every pair (benchmark, configuration) on its own, 6 at a time; per benchmark the fastest configuration counts.
- **Real portfolio**: all 7 configurations started together; the first sat/unsat wins.

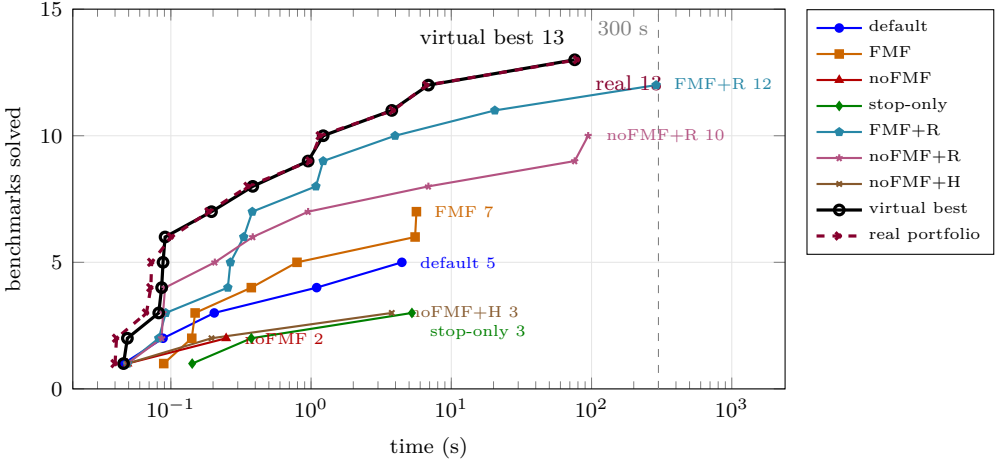
`scripts/portfolio.py` runs both modes; `scripts/slides/make_figures.py` makes the following tables and plots.

Every configuration on every benchmark (virtual portfolio)

benchmark	exp.	default	FMF	noFMF	stop-only	FMF+R	noFMF+R	noFMF+H	by	fastest
sc-1-th	unsat	timeout	unsat 0.79	timeout	timeout	unsat 0.38	**unsat 0.09**	timeout	3	0.09 s
sc-1-th-just-rf	unsat	unsat 0.20	unsat 0.15	**unsat 0.05**	unsat 5.24	unsat 0.09	unsat 0.05	unsat 0.05	7	0.05 s
sc-1-th-no-ltts	sat	**sat 0.05**	sat 0.09	unknown	sat 0.14	sat 0.05	unknown	timeout	4	0.05 s
sc-2-th	unsat	timeout	timeout	timeout	timeout	unsat 20.4	unsat 94.6	**unsat 3.76**	3	3.76 s
sc-2-th-just-rf	unsat	unsat 1.10	unsat 0.38	unsat 0.25	timeout	unsat 0.26	unsat 0.21	**unsat 0.20**	6	0.20 s
plsc-1-th	unsat	timeout	unsat 5.52	timeout	timeout	unsat 3.97	**unsat 0.95**	timeout	3	0.95 s
plsc-1-th-just-rf	unsat	timeout	timeout	timeout	timeout	unsat 0.33	**unsat 0.09**	timeout	2	0.09 s
plsc-1-th-no-ltts	sat	sat 0.09	sat 0.14	unknown	sat 0.38	**sat 0.08**	unknown	timeout	4	0.08 s
plsc-2-th	unsat	timeout	timeout	timeout	timeout	timeout	**unsat 75.8**	timeout	1	75.8 s
plsc-2-th-just-rf	unsat	timeout	timeout	timeout	timeout	unsat 1.08	**unsat 0.38**	timeout	2	0.38 s
tso-1-th	unsat	timeout	timeout	timeout	timeout	unsat 288.7	**unsat 6.87**	timeout	2	6.87 s
tso-1-th-just-rf	unsat	timeout	unsat 5.64	timeout	timeout	unsat 0.27	**unsat 0.09**	timeout	3	0.09 s
tso-1-th-no-ltts	sat	sat 4.44	timeout	unknown	timeout	**sat 1.22**	timeout	timeout	2	1.22 s
solved		5	7	2	3	12	10	3	**13 of 13**	

300 s per run. Bold: the fastest configuration, which the virtual best takes. “by”: how many configurations solve the file. 0 wrong answers. The best single configuration (FMF+R) solves 12; the portfolio solves **13 of 13**. plsc-2-th is solved by noFMF+R only.

The portfolio as a cactus plot

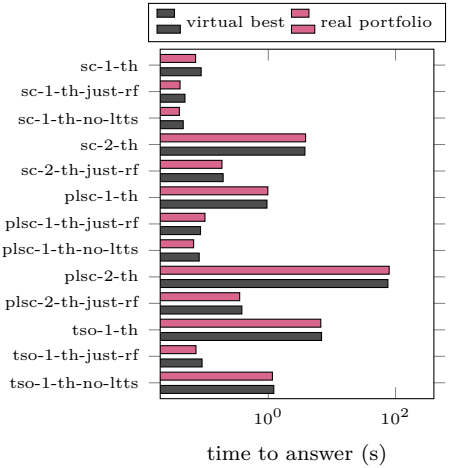


Each curve: one configuration's solve times, sorted; cumulative count; final count at its right end. Virtual best: per file the fastest configuration. Real: all 7 started together.

Virtual best versus real portfolio

benchmark	winner	VBS	real	CPU
sc-1-th	noFMF+R	0.09 s	0.07 s	0.5 s
sc-1-th-just-rf	noFMF / noFMF+H	0.05 s	0.04 s	0.3 s
sc-1-th-no-ltts	default	0.05 s	0.04 s	0.3 s
sc-2-th	noFMF+H	3.76 s	3.86 s	27.0 s
sc-2-th-just-rf	noFMF+H / noFMF+R	0.20 s	0.19 s	1.2 s
plsc-1-th	noFMF+R	0.95 s	0.98 s	6.8 s
plsc-1-th-just-rf	noFMF+R	0.09 s	0.10 s	0.7 s
plsc-1-th-no-ltts	FMF+R / default	0.08 s	0.07 s	0.5 s
plsc-2-th	noFMF+R	75.8 s	79.3 s	555.2 s
plsc-2-th-just-rf	noFMF+R	0.38 s	0.35 s	2.5 s
tso-1-th	noFMF+R	6.87 s	6.69 s	46.8 s
tso-1-th-just-rf	noFMF+R	0.09 s	0.07 s	0.5 s
tso-1-th-no-ltts	FMF+R	1.22 s	1.16 s	7.0 s

Virtual: 13 of 13. Real: 13 of 13, with the same winner on 10; real/virtual between 0.80 and 1.17 (median 0.95). Ratios below 1: the virtual runs also share the CPU (6 at a time).



Every real answer within 79.3 s. plsc-2-th needs 53 s on a fast core, 286 s on an efficiency core.

Evidence: how it was measured (`scripts/evidence`)

- A separate build (`scripts/evidence/rule-ids.patch`) gives every rule its own inference id (Alternate, Alt-Close, Chord, Distinct, Backward, Inclusion, Branch, Register, the five kinds of closure induction, the last-call completion, the lazy TC-Down) and an option `--rels-evidence-disable=IDS` that drops the lemmas of the given rules.
- **Usage**: every (benchmark, configuration) pair the portfolio solves (42 pairs, 13 benchmarks, 7 configurations) is rerun with `--stats-internal -o lemmas`: how many lemmas each rule sends, and real lemmas of each rule.
- **Ablation**: every pair is rerun once per rule that fired in it, with that rule disabled, and once per group of rules that can replace one another (all minimality rules, all kinds of closure induction): **763 runs**. Time limit max(60 s, 5× the original time); `--check-models` on the satisfiable files.
- Dropping lemmas can never give a wrong `unsat`; it can give a wrong `sat` (the rule is needed for the model to be correct) or lose the benchmark (the rule is needed for the proof, or to find it in time).

Which rule each benchmark needs (1/3): the cycle witness

rule	def	FMF	noFMF	stop	FMF+R	noFMF+R	noFMF+H
Cycle-Def	·	·		·	**tso1**	·	
Inst-Cycle	plsc1n, sc1r, sc1n, sc2r, tso1n	sc2r	**sc2r**	plsc1n, sc1r, sc1n	plsc2r, sc2, sc2r	**plsc2**, **plsc2r**, **sc2**, **sc2r**	**sc2**, **sc2r**
Unroll	plsc1n, sc1r, sc1n, sc2r, tso1n	plsc1n, sc2r	**sc2r**	plsc1n, sc1r, sc1n	plsc1n, plsc2r, sc2, sc2r	**plsc2**, **plsc2r**, **sc2**, **sc2r**	**sc2**, **sc2r**
Split-Len	sc2r	sc2r	**sc2r**	·	**tso1**, plsc2r, sc2, sc2r	**plsc2**, **plsc2r**, **sc2**, **sc2r**	**sc2**, **sc2r**
Alternate	·	·	·	·	**tso1**	·	·
Alt-Close	·	·	·	·	·	·	·
Chord	·	**sc2r**	**sc2r**	·	**plsc2r**, **sc2**, **sc2r**, **tso1**	**plsc2**, **plsc2r**, **sc2**, **sc2r**	**sc2**, **sc2r**
Distinct	**sc1r**	·	·	·	·	·	·
Backward		·	·		**sc2**	·	·
LC completion	**sc2r**	sc2r	**sc2r**	·	plsc2r, sc2, sc2r	**plsc2**, **plsc2r**, **sc2**, **sc2r**	**sc2**, **sc2r**
Branch	·			**sc1r**			
all minimality	**plsc1n**, **sc1n**, **tso1n**	**plsc1**, **plsc1n**, **sc1n**, **sc2r**	**sc2r**	·	**plsc1n**, **plsc2r**, **sc1n**, **sc2**, **sc2r**, **tso1**, **tso1n**	**plsc2**, **plsc2r**, **sc2**, **sc2r**	**sc2**, **sc2r**

Bold: without the rule the benchmark is **lost** (timeout or `unknown` within max(60 s, 5× the time)); red: **wrong** without it (`sat` on an unsat file, or a failed `--check-models`); ·: the rule fires but the benchmark is solved without it; empty: it does not fire. Benchmarks: plsc1r = plsc-1-th-just-rf, tso1n = tso-1-th-no-ltts, …

Which rule each benchmark needs (2/3): acyclicity, induction, Fun-UF

rule	def	FMF	noFMF	stop	FMF+R	noFMF+R	noFMF+H
Acyclic-Down	·	**sc1**, **sc1r**, **sc2r** plsc1, sc2r		**sc1r**	**sc1**, plsc1, plsc2r, sc2, sc2r	**plsc1**, **plsc2**, **plsc2r**, **sc1**, **sc2**, **sc2r**	**sc2**, **sc2r**
Register	·	·	·	·	·	·	·
Inclusion		**plsc1**, **sc1**, **tso1r**	·		**tso1**, **tso1r**	**tso1**, **tso1r**	·
TC-Sub: in TC(X)					**tso1**	·	
TC-Sub: R trans.					**sc2**, **tso1**	**plsc2**, **sc2**	**sc2**
TC-Sub: A×B					**sc2**, **tso1**	**plsc2**, **sc2**	**sc2**
TC-Sub: ∩/\					**plsc2r**	**plsc2**, **plsc2r**	
TC-Sub: R⊆S					**sc1n**, **tso1n**	·	·
Fun-UF	·	·	·	·	**sc2**, **tso1**, plsc2r	**plsc2**, **plsc2r**, **sc2**	·
all TC-Subset					**plsc2r**, **sc1n**, **sc2**, **tso1n**	**plsc2**, **plsc2r**, **sc2**	**sc2**

Bold: without the rule the benchmark is **lost** (timeout or `unknown` within max(60 s, 5× the time)); red: **wrong** without it (`sat` on an unsat file, or a failed `--check-models`); ·: the rule fires but the benchmark is solved without it; empty: it does not fire. Benchmarks: plsc1r = plsc-1-th-just-rf, tso1n = tso-1-th-no-ltts, ...

Which rule each benchmark needs (3/3): closures and joins

rule	def	FMF	noFMF	stop	FMF+R	noFMF+R	noFMF+H
TC-Down (lazy)					plsc1n, tso1n		
TC-Up	**sc2r**	plsc1	·	·	plsc1, plsc1r, plsc2r, tso1	**plsc1**, **plsc1r**, **plsc2r**, **tso1**	·
TC-Down	plsc1n, sc1n, tso1n	plsc1n	·	plsc1n, sc1n			
Join-Compose	sc1r, sc2r	**tso1r**, plsc1, sc1, sc1r, sc2r	**sc1r**, **sc2r**	sc1r	plsc1, plsc1r, plsc2r, sc1, sc1r, sc2, sc2r, tso1	**plsc1**, **plsc1r**, **plsc2**, **plsc2r**, **sc1**, **sc1r**, **sc2**, **sc2r**, **tso1**	**sc1r**, **sc2**, **sc2r**
Join-Split	tso1n	**tso1r**, plsc1, sc1	·	**sc1r**, plsc1n	**tso1**, plsc1, plsc1r, plsc2r, sc1, tso1r, tso1n	**plsc1**, **plsc1r**, **plsc2**, **plsc2r**, **sc1**, **sc2**, **tso1**, **tso1r**	

Bold: without the rule the benchmark is **lost** (timeout or `unknown` within max(60 s, 5× the time)); red: **wrong** without it (`sat` on an unsat file, or a failed `--check-models`); ·: the rule fires but the benchmark is solved without it; empty: it does not fire. Benchmarks: plsc1r = plsc-1-th-just-rf, tso1n = tso-1-th-no-ltts, ...

Not in the tables because they never fire on a solved pair: SAME-OPERANDS, the anchor conflict (`false` when every operand is inside $TC(S)$), FUN-PAIRS (only with `--rels-functional-mode=pairs`), NOT-FUN (no negated `rel.is-functional`), GROUND-TC, GROUND-JOIN.

A lemma of each rule, from a benchmark where it is needed (1/3)

rule	benchmark, configuration	a lemma it sent
Cycle-Def	tso1, FMF+R	`cyc(co) ⇒ (cyc117,cyc117) ∈ co+`
Inst-Cycle	sc1n, def	`cyc((rf ∪ (po ∪ (co ∪ fr)))) ⇒ cyclen6 ≥ 2`
Unroll	sc1n, def	`cyclen6 ≥ 2 ⇒ (cyc5,cyc9) ∈ rf+ ∨ (cyc5,cyc9) ∈ po+ ∨ (cyc5,cyc9) ∈ co+ ∨ (cyc5,cyc9) ∈ fr+`
Split-Len	sc2r, noFMF+H	`cyc((po ∪ rf)) ∧ cyclen10 ≥ 2 ⇒ cyclen10 ≥ 3 ∨ (cyclen10 = 2 ∧ cyc9 = cyc14)`
Alternate	tso1, FMF+R	`cyc((co ∪ (fr ∪ (rf ∪ (loc ∩ po+))))) ∧ cyclen8 ≥ 3 ⇒ ¬((cyc7,cyc10) ∈ co+ ∧ (cyc10,cyc95) ∈ co+)`
Alt-Close	sc1r, noFMF+H	`cyc((po ∪ rf)) ∧ cyclen6 = 3 ⇒ ¬((cyc9,cyc28) ∈ po+ ∧ (cyc5,cyc9) ∈ po+)`
Chord	sc2r, noFMF+H	`cyc((po ∪ rf)) ∧ cyclen10 ≥ 3 ⇒ (cyc9,cyc32) ∉ po+`
Distinct	sc1r, def	`cyclen6 ≠ 2 ∧ cyc((po ∪ rf)) ∧ cyclen6 ≥ 2 ⇒ cyc5 ≠ cyc9`
Backward	sc2, FMF+R	`cyc((rf ∪ (po ∪ (co ∪ fr)))) ∧ cyclen10 = 3 ⇒ (cyc68,cyc9) ∉ rf+`

Lemmas as cvc5 sent them (`-o lemmas`), with `let` expanded: `cyc`n, the witness elements; `cyclen`n, its length; `X+`, `rel.tclosure`. The benchmark is one where the rule is needed (else where it fires).

A lemma of each rule, from a benchmark where it is needed (2/3)

rule	benchmark, configuration	a lemma it sent
LC completion	sc2r, noFMF+H	`cyc((po ∪ rf)) ∧ cyclen10 ≥ 5 ⇒ cyc14 ≠ cyc82`
Branch	sc1r, stop	`cyc((po ∪ rf)) ∧ ¬(cyc(po)) ∧ cyclen7 = 3 ⇒ (cyc6,cyc10) ∉ po+ ∨ (cyc10,cyc49) ∉ po+`
Acyclic-Down	sc1, noFMF+R	`(cyc9,cyc5) ∈ po+ ∧ ¬(cyc(po)) ⇒ cyc5 ≠ cyc9`
Register	sc1r, noFMF+H	`¬(cyc(po)) ⇒ po+ = (po ∪ po+)`
Inclusion	tso1r, noFMF+R	`cyc(rf) ∧ ¬(cyc(po)) ⇒ cycedge21 ∈ rf ∧ cycedge21 ∉ po+`
TC-Sub: in TC(X)	tso1, FMF+R	`(cyc10,cyc95) ∈ ((loc ∩ po+))+ ⇒ (cyc10,cyc95) ∈ po+`
TC-Sub: R trans.	sc2, noFMF+H	`(cyc14,cyc9) ∈ co+ ∧ co = (co ∪ co;co) ⇒ (cyc14,cyc9) ∈ co`
TC-Sub: A×B	sc2, noFMF+H	`(cyc14,cyc9) ∈ fr+ ∧ Read×Write = (fr ∪ Read×Write) ∧ ∅ = (Read ∩ Write) ⇒ (cyc14,cyc9) ∈ fr`
TC-Sub: ∩/\	plsc2r, noFMF+R	`(cyc9,cyc14) ∈ ((loc ∩ po+))+ ∧ fun(address) ∧ loc = address;(address)`$^{-1}$ `⇒ (cyc9,cyc14) ∈ loc`

`cycedge`*n*: the fresh pair of INCLUSION; the closure-induction lemmas show the superset they use: `po+` (in $TC(X)$), `co` transitive, `Read×Write` with `Read` ∩ `Write` = ∅, `loc` (an intersection), `Event×Event` (an asserted superset).

A lemma of each rule, from a benchmark where it is needed (3/3)

rule	benchmark, configuration	a lemma it sent
TC-Sub: R⊆S	sc1n, FMF+R	`(cyc5,cyc9) ∈ po+ ∧ Event×Event = (po ∪ Event×Event) ⇒ (cyc5,cyc9) ∈ Event×Event`
Fun-UF	plsc2r, noFMF+R	`fun(address) ∧ (cyc9,srj23) ∈ address ⇒ srj23 = (rfun_34 cyc9)`
TC-Down (lazy)	plsc1n, FMF+R	`(cycedge59.0,cycedge59.1) ∈ po+ ⇒ (cycedge59.0,cycedge59.1) ∈ po ∨ ((cycedge59.0,stc181) ∈ po ∧ (stc282,cycedge59.1) ∈ po ∧ (stc181 = stc282 ∨ (stc181,stc282) ∈ po+))`
TC-Up	plsc1r, noFMF+R	`(cyc6,cyc10) ∈ (loc ∩ po+) ∧ (loc ∩ po+) = (loc ∩ po+) ⇒ (cyc6,cyc10) ∈ ((loc ∩ po+))+`
TC-Down	sc1n, def	`(cyc5,cyc9) ∈ po+ ⇒ (cyc5,cyc9) ∈ po ∨ ((cyc5,stc123) ∈ po ∧ (stc224,cyc9) ∈ po ∧ (stc123 = stc224 ∨ (stc123,stc224) ∈ po+))`
Join-Compose	sc1r, noFMF+H	`(cyc9) ∈ Write ∧ (cyc9,cyc5) ∈ rf+ ∧ rf = rf+ ⇒ (cyc5) ∈ Write;rf`
Join-Split	sc1, noFMF+R	`(cyc5,cyc9) ∈ (address;(address)`$^{-1}$` ∪ (co ∪ (rf ∪ fr))) ∧ address;(address)`$^{-1}$` = (address;(address)`$^{-1}$` ∪ (co ∪ (rf ∪ fr))) ⇒ (cyc5,srj22) ∈ address`

`rfun`n: the skolem function of Fun-UF; `stc`n, `srj`n: the fresh elements of TC-Down and Join-Split.

What the evidence says

- **Essential**: the witness (Inst-Cycle, Unroll, Split-Len, the last-call completion): without each, 7-8 (benchmark, configuration) pairs are lost and 4-14 answered **wrongly** (no cycle behind `(rel.cyclic R)`). Acyclic-Down: 13 pairs lost, 6 wrong. Chord: 12 lost.
- **Needed, in specific configurations**: Inclusion (7: FMF, FMF+R, noFMF+R); closure induction (8 as a group, every kind needed somewhere, only in the R configurations); Fun-UF (5: FMF+R, noFMF+R); Alternate, Distinct, Backward, Branch, Cycle-Def (one benchmark each). All minimality rules together: 21, including the satisfiable files (minimality also guides the model search).
- **Redundant on these benchmarks**: Alt-Close (fires on 37 pairs, never needed alone; the other minimality rules cover it), Register (fires on 42, never needed: the closure terms it registers exist anyway in these benchmarks).
- **Never fire**: Same-Operands, the anchor conflict, Fun-Pairs, Not-Fun, Ground-TC, Ground-Join. The hammer solves its 3 benchmarks *without* a grounding conflict: its gain is not splitting closures and joins.
- **Needed for correct models only**: lazy and eager TC-Down (wrong models of the `no-ltts` files without them, never needed for unsat).
- **Existing rules**: Join-Compose (15 lost, 15 wrong), Join-Split (11, 10), TC-Up (5, 5).

Caveats: one rule at a time underestimates rules that replace one another (hence the groups); "lost" means not within 5× the time.

Testing

- **Regressions**: 53 new or changed files; all 443 sets, relations and bags regressions and all 2800 `regress0` tests pass with assertions on.
- **Proofs**: the CPC proofs of all 84 applicable unsat relations regressions check with ethos; `garbage_collect.cvc` times out, as on main. The check found that `rel.rclosure`, `rel.rtclosure` were missing from the CPC signature (added).
- **Fuzzing** against brute force: 5 500 random problems (`rel.cyclic` of unions, nested either way, `(not (rel.cyclic S))`, pairs forced in, out, or free) under five option sets: no wrong answer, no error.
- **Bugs found on the way, fixed in this PR**:
  - the join grounding conflict refuted satisfiable branches under the hammer (missing distinctness);
  - `rel.rclosure` of a union crashed debug builds, and of a variable answered `sat` wrongly (now `unknown`);
  - the MiniSat default answered `unsat` wrongly with `:produce-interpolants` (a latent issue, also on main with `--sat-solver=minisat`); the default now skips sygus-style problems.
- **CI**: formatting (clang-format 22, whole files) and the CPC/Logos check pass.

Contributions

- Two predicates for relational specifications, `rel.cyclic` and `rel.is-functional`, with API, parser, printer, type rules, rewrites, CPC declarations and documentation.
- A calculus for them in the relations solver:
  - a cycle witness that is a shortest cycle of the operands' closures, unrolled one element per round and completed from the model;
  - acyclicity, and anchors that connect cyclic and acyclic constraints;
  - closure induction, which proves closure memberships without fresh elements;
  - functionality by a skolem function;
  - lazy and grounded closure reasoning.
- A strategy that places each rule: context rules at full effort, model-dependent rules at last call.
- On the 13 litmus-template benchmarks: **13 of 13** solved by the portfolio (virtual and real), no wrong answer.

Where to find it

- PR to cvc5 main: `mudathirmahgoub/cvc5#17` (branch `cycle-ext-cyclic`); the same branch is `rachelcleaveland/cvc5#9`, stacked on the earlier `cycle-ext` PRs.
- Benchmarks: `rachelcleaveland/relational-solver-benchmarks`, directories `benchmarks-ltts` and `benchmarks-ltts-nofmf`.
- Temporary, to be deleted before merging: `scripts/` (portfolio runner, ablation, comparison, converters) and `scripts/slides/` (this deck, its figure generator and the run data).
- Rebuild the deck: `cd scripts/slides; python3 make_figures.py data/final-300s figures; pdflatex relations-pr.tex`.